

# Examples Guide

## Examples Guide

gemstone-js ships a small installed example catalog. The catalog is meant to serve two jobs: quick local discovery from an installed package, and stable file paths that release checks can verify before publishing.

### Discovery

```
gemstone-js-examples
gemstone-js-examples --json
gemstone-js-examples --kind web
gemstone-js-examples --commands
gemstone-js-examples --plans
```

Use `--show <name>` to print a packaged example and `--path <name>` to resolve its installed file path:

```
gemstone-js-examples --show quickstart
gemstone-js-examples --path web-fetch
```

The command list intentionally includes only examples that can be run directly. Reference fixtures such as generated wrapper output and route-handler modules stay visible in the catalog but do not get direct run commands.

Installed-package smoke checks should resolve examples through the CLI instead of assuming checkout-relative paths:

```
gemstone-js-examples --show quickstart
gemstone-js-examples --show codegen-manifest
gemstone-js-examples --show booking-decorators
```

### Guided Plans

The plan view groups examples by workflow:

```
gemstone-js-examples --plan first-session
gemstone-js-examples --commands --plan data-persistence
gemstone-js-examples --json --plan web-service
```

Current plans:

- **first-session**: connect, run the MagLev branch compatibility session example, evaluate, and write/read ObjectLog entries.
- **data-persistence**: roots, dictionaries, query helpers, bulk selector sends, transparent object mapping, GStore, and migrations.
- **typed-codegen**: manifests, decorated source, and generated wrappers.
- **web-service**: Fetch, local explorer, route-handler, Express, Fastify, and Hono shapes.
- **ops-release**: migration, ObjectLog, and codegen checks used before release.

### Environment

Live examples use the same `GS_*` environment as `Session.configFromEnv()`:

```
export GS_STONE=gs64stone
export GS_NETLDI=netldi
export GS_HOST=localhost
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

The canonical JavaScript names above take precedence. Existing Pharo bridge shells can use `GS_USER`, `GS_PASS`, `GS_NETLDI_HOST`, `GS_NETLDI_NAME_OR_PORT`, and `GS_SERVICE` as compatibility aliases. Set `GS_NATIVE_SESSION_WORKER=1` when you want examples that call `Session.connect()` to use the optional Node `GciSessionWorker` backend instead of the raw `Gci` backend.

The committed check path does not connect to GemStone:

```
npm run examples:check
npm run live:check
```

That check syntax-checks TypeScript examples, parses JSON examples, validates catalog metadata, validates guided plans, and fails if an example file is not in the catalog. The live-smoke guard does not connect to GemStone; it runs the `tests/live.test.ts` skip path and verifies that the opt-in live regression keeps covering connection setup, arrays, dictionaries, globals, persistent roots, GStore, query helpers, and migrations.

Use `npm run test:live:worker` to run the same opt-in live suite with `GS_NATIVE_SESSION_WORKER=1`.

## MagLev Branch Example

`examples/maglev-branch-usage.ts` mirrors the session examples from `GemStone-Pharo-Bridge/doc/MAGLEV-BRANCH-USAGE.m` in JavaScript form. It uses `GbsSessionParameters`, `session.userGlobals`, `session.bridgeRoot`, `session.commit()`, `session.commitTransactionOrSignalConflict()`, and `session.disconnect()` so existing GemStone-Pharo-Bridge users can see the closest equivalent shape before moving to lower-level `Session` and `PersistentRoot` APIs.

## Codegen Examples

The generated wrapper examples are committed so they can be reviewed like normal source:

```
npm run codegen:check
npm run codegen:scan:check
```

`examples/codegen.manifest.json` demonstrates typed imports, typed arguments, array and dictionary argument marshalling, value returns, raw OOP returns, and retained typed-object returns. `examples/booking.decorators.ts` exercises the decorator scanner and emits `examples/booking.decorators.generated.ts`.

For the broader object-mapping model, including `TypedOop<T>` handles, generated selector wrappers, dictionary payload conversion, and value converters, see `docs/object-mapping.md`.

`examples/object-mapping.ts` shows `mappedObject()` with async property-style methods, setter selectors, object selectors, and snapshots. The guide expands that example into a mapping decision table, explicit selector configuration, relationship handle mapping, dictionary-backed snapshot fields, and a production checklist for live Stone usage. It also documents root/global mapping, retained-handle lifetime rules, and the migration path from direct `TypedOop<T>.send()` calls to proxy options and generated `*Ref` classes.

`examples/transparent-object-mapping.ts` shows the higher-transparency proxy: `await booking.status`, callable selector accessors, relationship handles, queued writes through `$assign()/$flush()`, bounded snapshots, and request-scoped identity reuse through the same mapping API.

`examples/object-mapping.manifest.json` is the first committed mapping manifest. It describes a `BookingRef`, selector methods, a setter, bounded snapshot fields, and a repository method returning a typed ref. The manifest is schema-backed by `schemas/object-mapping-manifest.schema.json`; generated `BookingRef` output is the next step.

`examples/smalltalk-bridge.ts` shows the Python-style dynamic bridge: `smalltalkBridge(session)`, lazy global proxies, awaitable selector properties, `st.Array.new_(3)` underscore-to-colon selector dispatch, exact `$send*()` controls, and selector object results wrapped directly as transparent proxies.

The next planned examples should cover generated `BookingRef`-style classes around `TypedOop<T>`, repository helpers returning typed refs, and bounded snapshot/dictionary payload helpers. Those examples should remain generated and committed so mapping behavior is reviewable in CI.

## Web Examples

The dependency-free Fetch example is the smallest web service shape:

```
node --experimental-strip-types examples/web-fetch.ts
```

The dependency-free browser explorer is a small local workbench for status, doctor output, OOP inspection, roots/globals, workspace eval, class browsing, and generated-wrapper preview:

```
node --experimental-strip-types examples/explorer.ts
```

Framework examples show the same request/session lifecycle through common Node frameworks:

```
npm install express
```

```
node --experimental-strip-types examples/web-express.ts
```

```
npm install fastify
```

```
node --experimental-strip-types examples/web-fastify.ts
```

```
npm install hono @hono/node-server
```

```
node --experimental-strip-types examples/web-hono.ts
```

Route-handler frameworks can start from `examples/web-route-handler.ts`, which exports `GET()` and `POST()` functions around the Fetch adapter.